



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure (P), Environment (E), Actuators (A), Sensors (S)

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width`, `self.height`, `self.walls`, `self.food_positions`, `self.opponents` and `self.agent_pos`. These describe the physical grid world. `score`, `steps` and `collision` are bookkeeping, not the physical world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-agent (competitive). `self.opponents` holds other entities that move on their own each turn. Bumping into one costs `-50` and ends the game, so the agent's score depends on their actions as well as its own. (The `__main__` block passes `num_opponents = 0`, and that setup would be single agent.)

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, growing history of every percept the agent has received, from the start up to now. The agent's choice can only depend on this history, never on hidden data or future events.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S). It is the channel through which the agent receives its percepts.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially observable. The agent sees its own position, the opponents' positions, the score and how many food items are left. It does not see where the food is: `smells_food` is only true when the agent is already standing on it. It also gets no map of the walls. So important parts of the state are hidden.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P). `self.score` is the performance measure being updated

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

the measure should reward real results in the world, not the machine's busywork. If it scored internal effort, the agent could farm points without doing the task, as in the vacuum exploit (Q11). Scoring external events like hitting a wall means the only way to score well is to actually behave well.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes more partially observable. Traps are now a real part of the state that affects the score, but the agent has no sensor for them. It can only notice one after stepping on it and losing points.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

- Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding 'smells_toxin' , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

smells_toxin puts traps into the percept sequence. Rationality means maximising expected utility based on what the agent has perceived. With this sensor, the agent can learn that a cell is dangerous and avoid going back to it. Without it, the agent could not learn this and would keep going back to it.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

- 1. Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In the vacuum world exploit, a vacuum earned +1 each time it sucked up dirt, so it learned to dump the dirt back out and clean the same spot forever for endless points. The fix was to reward how long the rooms stay clean instead.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING